Documentación técnica v0.2.0

Gestionado y desarrollado por Santino Ventrice

Fecha de inicio: 26/10/2026

1. Visión y problema

Muchas heladerías pequeñas controlan su producción y su stock de forma manual, con planillas, cuadernos o mensajes de WhatsApp. Esto genera tres problemas frecuentes: no se sabe con precisión cuánto helado queda de cada sabor, los insumos se terminan sin aviso, y los lotes vencen sin que nadie lo registre a tiempo.

Este sistema busca centralizar la producción, las ventas y el stock de una heladería en una sola aplicación. Su objetivo es que el encargado sepa qué se produjo, qué se vendió y qué queda, y que el vendedor vea en el mostrador qué sabores tiene disponibles.

El sistema está pensado para el dueño, el encargado de producción y el personal de venta de una heladería chica. Su valor se mide en menos faltantes, menos mermas por vencimiento y menos tiempo dedicado a cargar planillas.

2. Alcance

Versión v1.0

- **Producción:**
    - Registro de tandas con descuento de insumos según receta
    - Alertas de stock mínimo por sabor
    - Control de vencimiento por lote e historial de cambios de recetas.
- **Ventas:**
    - Registro de ventas por sabor y medio de pago
    - Descuento automático de stock al vender
    - Cierre de caja por turno.
- **Usuarios y reportes:**
    - Roles de admin, encargado y vendedor
    - Reporte de producción vs. ventas
    - Panel de inicio con el resumen del día
    - Exportación a CSV

3. Usuarios y roles

Admin (dueño o responsable general)

- Ve y configura todo: sabores, insumos, recetas y precios.
- Crea y desactiva usuarios, y asigna roles.
- Accede a todos los reportes, incluido el de producción vs. ventas, y exporta a CSV.
- Puede corregir o anular tandas y ventas, siempre dejando registro del cambio.

Encargado (quien supervisa la producción y el turno)

- Registra las tandas de producción y controla el stock de insumos y sabores.
- Gestiona los lotes y sus vencimientos, y recibe las alertas de stock mínimo.
- Modifica recetas, pero el historial guarda quién hizo cada cambio.
- Ve reportes de producción, pero no los datos de caja ni la gestión de usuarios.

Vendedor (quien atiende en el mostrador)

- Registra ventas por sabor y medio de pago.
- Abre y cierra su turno de caja, y carga la diferencia si hay.
- Ve el stock disponible de cada sabor para saber qué puede vender.
- No puede modificar recetas, tandas ni precios, ni ver reportes de producción.

4. Modelo de datos

Entidades principales

Entidad	Descripción	Campos clave
usuarios	Perfil de cada persona, ligado a Supabase Auth	id, nombre, rol, activo
presentaciones	Formas de venta: vasito, cucurucho, cuarto, kilo	id, nombre, litros_equivalentes, activo
sabores	Productos que se venden	id, nombre, activo

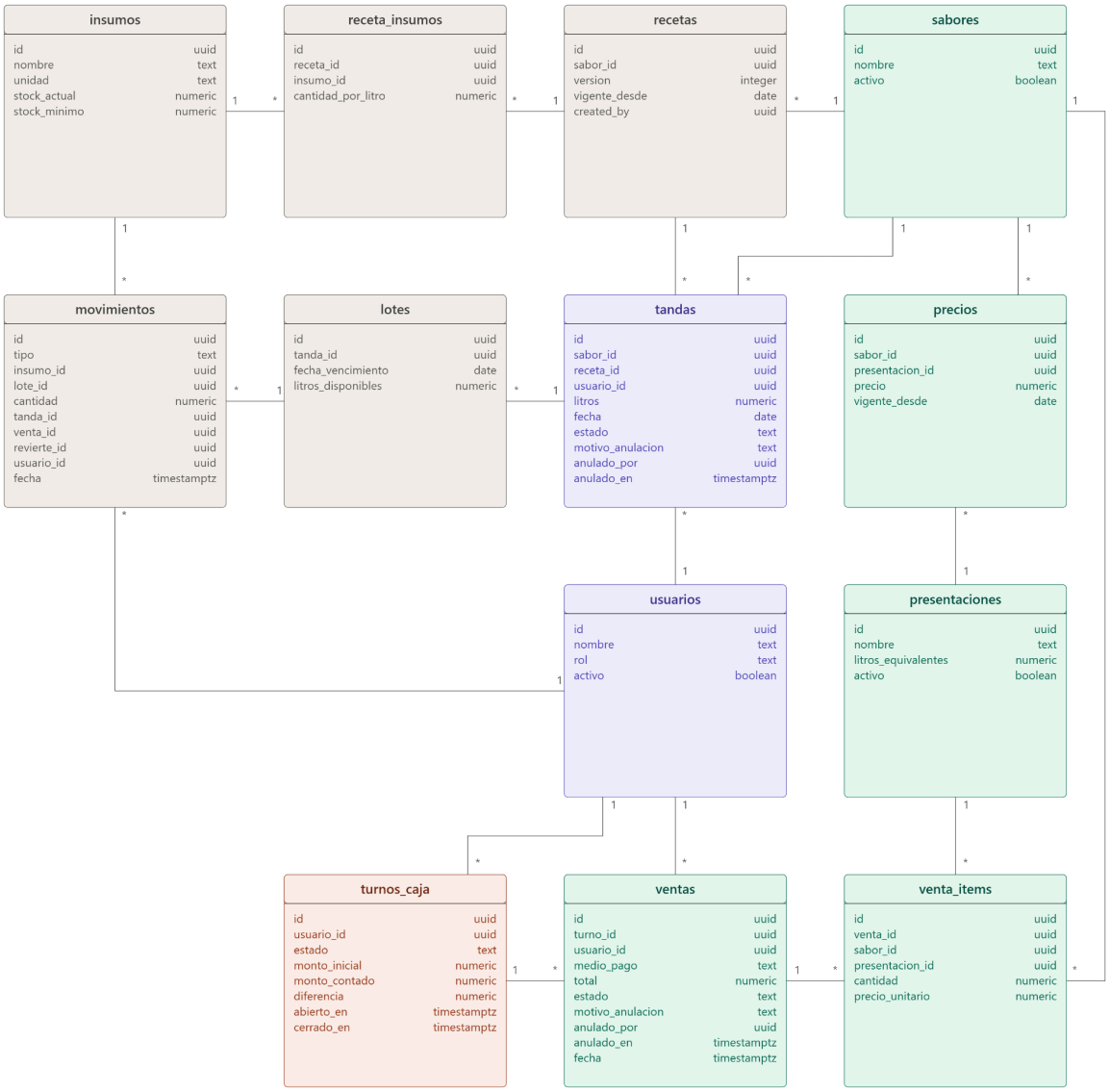
precios	Precio de cada sabor por presentación, con historial	id, sabor_id, presentacion_id, precio, vigente_desde
insumos	Ingredientes y materias primas	id, nombre, unidad, stock_actual, stock_minimo
recetas	Receta de un sabor, versionada	id, sabor_id, version, vigente_desde, created_by
receta_insumos	Insumos y cantidades de cada receta	id, receta_id, insumo_id, cantidad_por_litro
tandas	Producción de un sabor, con posibilidad de anulación	id, sabor_id, receta_id, usuario_id, litros, fecha, estado, motivo_anulacion, anulado_por, anulado_en
lotes	Parte de una tanda con fecha de vencimiento	id, tanda_id, fecha_vencimiento, litros_disponibles
turnos_caja	Apertura y cierre de caja por turno	id, usuario_id, estado, monto_inicial, monto_contado, diferencia, abierto_en, cerrado_en
ventas	Cabecera de cada venta, con posibilidad de anulación	id, turno_id, usuario_id, medio_pago, total, estado, motivo_anulacion, anulado_por, anulado_en, fecha
venta_items	Detalle de cada venta, con el precio del momento	id, venta_id, sabor_id, presentacion_id, cantidad, precio_unitario
movimientos	Libro de stock: cada cambio con su origen. Nunca se edita ni borra	id, tipo, insumo_id o lote_id, cantidad, tanda_id, venta_id, revierte_id, usuario_id, fecha

Relaciones

- Un **sabor** tiene muchas **recetas** a lo largo del tiempo, pero solo una vigente, y muchos **precios**, uno por cada presentación.
- Una **receta** tiene muchos **insumos** con su cantidad por litro.
- Una **tanda** pertenece a un sabor y usa la receta vigente en ese momento.
- Una **tanda** genera uno o más **lotes**.
- Una **venta** pertenece a un **turno de caja** y tiene uno o más **venta_items**. Cada ítem referencia un sabor y una presentación.
- Un **movimiento** referencia un **insumo** o un **lote**, y opcionalmente la **tanda** o la **venta** que lo originó. Una reversa apunta al movimiento que revierte (`revierte_id`).

Reglas de negocio

1. **Una tanda descuenta insumos según la receta vigente al momento de producirla.** Si después cambia la receta, las tandas anteriores no se modifican.
2. **Una venta descuenta litros de sabor de los lotes disponibles, siguiendo el criterio FEFO** (primero en vencer, primero en salir). No se venden lotes vencidos.
3. **Las anulaciones no borran registros.** Se crea un movimiento de reversa y se marca la operación como anulada, con el usuario que la hizo.
4. **Cada cambio de receta crea una nueva versión.** Así el historial de cambios queda completo y auditable.
5. **Cada turno de caja es único por usuario y no puede cerrarse con ventas abiertas.**
6. **Cuando el stock de un insumo cae por debajo de su stock mínimo, se genera una alerta.** La alerta no bloquea la operación, sólo la señala.
7. **El stock de un sabor es la suma de los litros disponibles en sus lotes no vencidos.**



5. Flujos principales

5.1 Registrar una tanda de producción

1. El encargado selecciona el sabor a producir. El sistema carga la receta vigente.
2. Ingresa los litros producidos y la fecha de vencimiento del lote.
3. El sistema calcula los insumos necesarios (cantidad por litro × litros) y verifica que haya stock suficiente.
4. Si alcanza, crea la tanda, genera el lote y registra un movimiento de consumo por cada insumo.
5. Si un insumo queda por debajo de su stock mínimo, se muestra la alerta.

- **Error:** si falta algún insumo, la tanda no se guarda y el sistema indica cuál y cuánto.

5.2 Registrar una venta

1. El vendedor tiene un turno de caja abierto. Si no, el sistema le pide abrirlo.
2. Selecciona sabor, tamaño y cantidad. El sistema muestra el stock disponible del sabor (suma de lotes no vencidos).
3. Elige el medio de pago y confirma.
4. El sistema descuenta los litros de los lotes en orden FEFO, registra un movimiento por cada lote afectado y guarda la venta con sus ítems.

- **Error:** si no hay stock suficiente, la venta no se confirma.

5.3 Cerrar el turno de caja

1. El vendedor ingresa el monto contado en caja.
2. El sistema calcula el total esperado (monto inicial + ventas en efectivo) y muestra la diferencia.
3. Se guarda el cierre con la diferencia. El turno queda cerrado y no puede volver a usarse.

- **Regla:** no se puede cerrar el turno si hay ventas en estado pendiente.

5.4 Anular una venta o una tanda

1. El admin o el encargado selecciona el registro y escribe el motivo de la anulación.
2. El sistema marca el registro como anulado, guarda quién lo hizo y la fecha.
3. Crea movimientos de reversa para devolver el stock. Los registros originales no se borran.

5.5 Modificar una receta

1. El admin o el encargado edita la receta de un sabor.
2. El sistema crea una nueva versión con fecha de vigencia desde ese momento.
3. Las tandas anteriores conservan la receta con la que se produjeron.

5.6 Alerta de stock mínimo

1. Cada vez que un movimiento reduce el stock de un insumo, el sistema compara el saldo con el stock mínimo.
2. Si el saldo queda por debajo, se genera una alerta visible en el panel de inicio.
3. La alerta se cierra cuando el stock vuelve a superar el mínimo, por una nueva compra o ajuste.

5.7 Ver el reporte de producción vs. ventas

1. El admin o el encargado elige un rango de fechas y, opcionalmente, un sabor.
2. El sistema muestra litros producidos, litros vendidos y litros vencidos o descartados por período.
3. Desde ese reporte se puede exportar a CSV.

6. Arquitectura

La aplicación es una sola base de código en Next.js con TypeScript. Toda la persistencia, la autenticación y la seguridad viven en Supabase, así que no hay un backend separado que mantener. Las operaciones que modifican stock se ejecutan como funciones dentro de Postgres, para que cada producción o venta sea una transacción completa: o se registra todo, o no se registra nada.

Stack

Capa	Tecnología	Motivo
Frontend y servidor	Next.js (App Router) con TypeScript	Renderizado en servidor, rutas y acciones del servidor en el mismo proyecto
Base de datos	Postgres vía Supabase	Relaciones, transacciones y tipos `uuid` y `numeric` del modelo
Autenticación	Supabase Auth	Login con email y contraseña, sin implementar sesiones a mano
Autorización	Row Level Security (RLS) en Supabase	Las reglas de rol se aplican en la base, no solo en la interfaz
Validación	Zod	Mismo esquema para formularios y para las acciones del servidor
Estilos	Modules CSS	Cada página o componente tiene su <nombre>.module.css
Tests unitarios	Vitest	Lógica de stock, FEFO y costos, que es donde hay más riesgo

Tests de flujos	Playwright	Recorre registrar una tanda, una venta y el cierre de caja
CI/CD	GitHub Actions	Corre lint, tests y build en cada pull request
Deploy	Vercel	Preview por pull request. Deploy de `development` a staging y de `main` a producción con aprobación.

Estructura del repositorio

```
gelato-track/
├── app/                  # Rutas de Next.js (login, panel, tandas, ventas, caja, reportes)
├── components/           # Componentes de interfaz reutilizables
├── lib/
│   ├── supabase/         # Clientes de Supabase para servidor y navegador
│   ├── stock/            # Lógica pura: consumo de insumos, FEFO, alertas de mínimo
│   ├── costos/           # Cálculo de costos por litro (si se suma más adelante)
│   └── validaciones/     # Esquemas de Zod
├── supabase/
│   ├── migrations/       # Cambios de esquema versionados y las funciones de negocio
│   ├── config.toml       # Configuración de la CLI de Supabase
│   └── seed.sql          # Datos de catálogo para desarrollo local
├── tests/
│   ├── unit/             # Vitest sobre lib/
│   └── e2e/              # Playwright sobre los flujos principales
├── docs/                 # Documentación del proyecto (este documento y los ADR)
└── .github/workflows/    # Pipeline de CI/CD
```

La carpeta `lib/stock/` contiene funciones puras, sin acceso a la base de datos. Esto permite testearlas con Vitest sin levantar nada, y la misma lógica se usa tanto en la validación previa como en la confirmación de la operación.

Decisiones clave

Operaciones críticas como funciones de Postgres. Registrar una tanda, una venta o el cierre de caja involucra varias tablas a la vez: crear la tanda, generar lotes, registrar movimientos y actualizar stock. Si esto se hace desde la aplicación en varias llamadas, un fallo a mitad de camino deja datos inconsistentes. Por eso cada una de estas operaciones es una función de Postgres que se ejecuta en una sola transacción, y la aplicación solo la invoca.

Seguridad en la base, no en la interfaz. Las políticas de RLS definen qué puede leer y escribir cada rol. Por ejemplo, un vendedor puede insertar ventas en su turno abierto, pero no puede modificar tandas ni recetas. Esto protege los datos aunque alguien llame a la API directamente.

Anulaciones sin borrado. Como establece la regla de negocio, ningún registro se elimina. Las anulaciones actualizan un estado y crean movimientos de reversa, así que la tabla de `movimientos` siempre es un historial completo.

Recetas versionadas. Cada tanda guarda referencia a la receta vigente en el momento de producirla. Cambiar una receta crea una nueva versión y no altera tandas anteriores.

Sin backend separado. Las acciones del servidor de Next.js cubren los formularios y la lógica de aplicación. No hay API pública en la versión 1, lo que reduce superficie de ataque y trabajo de mantenimiento.

7. Entornos y despliegue

El proyecto tiene cuatro entornos, cada uno con un propósito claro y con sus propios datos.

Entorno	Dónde corre	Base de datos	Datos	Quién lo usa
Local	Tu máquina	Supabase local (Docker)	Datos de prueba `seed.sql`	Cualquiera
Preview	Vercel, una URL por pull request	`gelato-track-staging`	Datos de prueba	Rama de cada PR
Staging	`gelato-track-staging.vercel.app`	`gelato-track-staging`	Datos de prueba	Rama `development`
Producción	`gelato-track.vercel.app`	`gelato-track-prod`	Datos reales	Rama `main`, con aprobación manual

Local

Cada desarrollador levanta Supabase en local con Docker y aplica las migraciones de la carpeta `supabase/migrations/`. Con `npx supabase db reset` la base queda idéntica en estructura a la de staging y se cargan los datos de `supabase/seed.sql`. El seed nunca se ejecuta con `db push`, así que staging y producción no reciben datos de prueba por accidente. Nunca se conecta el entorno local a la base de staging ni a la de producción.

Preview

Cada pull request genera un preview automático en Vercel. El preview apunta a la base de staging, así que los cambios se prueban con datos de prueba y nunca con datos reales. Cuando el pull request se cierra, el preview se elimina.

Staging

Staging es el entorno de validación final y el destino del deploy de la rama `development`. Tiene su propio proyecto de Supabase y un dominio fijo en Vercel asignado a esa rama. Antes de cualquier promoción a producción, los flujos principales (registrar una tanda, una venta y el cierre de caja) tienen que pasar en staging, y las migraciones pendientes se aplican acá primero.

Producción

Producción tiene su propio proyecto de Supabase, con datos reales y copias de seguridad automáticas activadas. Su dominio corresponde a la rama `main`. Ningún entorno inferior tiene credenciales para escribir en ella.

Modelo de ramas

El repositorio tiene dos ramas principales, con funciones distintas:

- **`development`** es la rama de integración. Las tareas se mergean acá y cada merge despliega a staging.
- **`main`** es producción. Solo recibe cambios desde `development`, mediante un pull request de promoción.

Las ramas de tarea (`feat/`, `fix/`, `chore/`, `ci/`, `test/`, `docs/`) salen de `development` y vuelven a `development` por pull request con *Squash and merge*. Los hotfixes salen de `main`, se mergean a `main` y luego `main` se trae de vuelta a `development`.

Flujo de promoción a producción

1. Se abre un pull request desde una rama de tarea hacia `development`. El pipeline de CI ejecuta lint, tipos, tests unitarios, tests de base de datos y build.
2. Al hacer merge a `development`, el job de staging aplica las migraciones pendientes en `gelato-track-staging` y Vercel publica la versión en `gelato-track-staging.vercel.app`.
3. Playwright corre los tests de flujos contra staging. Si fallan, el despliegue se detiene y el cambio no avanza.
4. Cuando el cambio está validado en staging, se abre un pull request de `development` a `main`. Los checks de CI tienen que pasar.
5. Al hacer merge a `main`, el job de producción queda en espera de aprobación manual. Está configurado como un *environment* de GitHub llamado `production`, con revisores requeridos, así que nadie puede desplegar sin que alguien con permisos lo apruebe desde la pestaña Actions.
6. Una vez aprobado, se aplican las migraciones pendientes en `gelato-track-prod`, se ejecuta el smoke test contra `gelato-track.vercel.app` y Vercel publica la nueva versión.

Tener aprobación manual significa que el deploy a producción siempre es una decisión consciente, y que producción nunca recibe código que no pasó por staging.

Variables y secretos

Los secretos se guardan en GitHub, nunca en el repositorio, y se organizan por alcance:

- **Repositorio:** `SUPABASE_ACCESS_TOKEN`, que usa la CLI de Supabase en todos los entornos.
- **Environment `staging`:** `SUPABASE_DB_PASSWORD` y `SUPABASE_PROJECT_REF` del proyecto de staging, y la variable `STAGING_URL`.
- **Environment `production`:** `SUPABASE_DB_PASSWORD` y `SUPABASE_PROJECT_REF` del proyecto de producción, la variable `PROD_URL`, y los secrets `SMOKE_USER_EMAIL` y `SMOKE_USER_PASSWORD` del usuario de prueba de solo lectura.

En Vercel solo se configuran las variables públicas de la aplicación (`NEXT_PUBLIC_SUPABASE_URL` y la clave publicable), por entorno: Preview y Development apuntan a staging, y Production apunta a producción. Las credenciales de la CLI y las contraseñas de base de datos no se cargan en Vercel. El archivo `.env.example` documenta las variables sin valores reales, y el `.env` local está excluido de Git.

Datos de prueba

Los datos de prueba se cargan con `supabase/seed.sql` en local, y con un script equivalente en staging. El seed inserta presentaciones, sabores, insumos, recetas y precios, y usa `gen_random_uuid()` para los ids. Los usuarios de prueba se crean desde Supabase Auth y se activan con su rol. Como medida de seguridad, el script de staging valida el nombre del proyecto de Supabase antes de insertar, y se detiene si detecta `prod`.

Rollback

Si un despliegue a producción sale mal, se vuelve al commit anterior desde Vercel, que tarda segundos. Si el cambio incluía una migración, no se revierte a mano: se crea una nueva migración correctiva que deshace el efecto, y se aplica con el mismo flujo de aprobación. Las migraciones aplicadas en producción nunca se editan (ADR-014).

8. Calidad y pruebas

La estrategia de pruebas apunta a los lugares donde un error cuesta plata: el cálculo de stock, las ventas, el cierre de caja y los permisos por rol. El resto del código se prueba con menos profundidad.

Niveles de prueba

Nivel	Herramienta	Qué cubre	Dónde corre
Unitarias	Vitest	Funciones puras de `lib/stock/` y `lib/validaciones/`	Local y CI, en cada pull request
Base de datos	Vitest con Supabase local	Funciones de Postgres (`registrar_tanda`, `registrar_venta`, `cerrar_turno`) y políticas de RLS	Local y CI
Flujos (end to end)	Playwright	Recorridos completos desde la interfaz	CI contra staging, y local bajo demanda
Manual	Revisión humana	Usabilidad, textos y casos que no conviene automatizar todavía	Antes de cada promoción a producción

Qué se testea en cada nivel

Unitarias. Son el grueso de los tests, porque son rápidas y la lógica es densa. Cubren el cálculo de consumo de insumos según receta y litros, la selección de lotes en orden FEFO, la generación de alertas de stock mínimo, el cálculo de la diferencia de caja y las validaciones de Zod. Cada regla de negocio de la sección 3 tiene al menos un test que la verifica.

Funciones de Postgres y RLS. Acá se prueba lo que no puede fallar silenciosamente. Se verifica que una tanda con insumos insuficientes no deje nada guardado a medias, que una venta sin stock sea rechazada, que no se pueda cerrar un turno con ventas pendientes, y que cada rol vea y modifique solo lo que le corresponde. Por ejemplo, un vendedor intentando editar una receta tiene que recibir un error, y no un registro modificado.

Flujos con Playwright. Son pocos pero cubren lo que más importa: registrar una tanda y ver el stock actualizado, registrar una venta desde el mostrador y verificar que descuente del lote correcto, y abrir y cerrar caja con una diferencia. Se corren contra staging en cada pull request que toque una de estas rutas.

Manual. Antes de cada promoción a producción se hace una revisión corta sobre staging: que los textos estén en español correcto, que los estados se entiendan sin explicación y que la exportación a CSV abra bien en una planilla.

Cobertura

No se fija un porcentaje global de cobertura, porque suele incentivar tests que no prueban nada. En cambio se exige que `lib/stock/` y las funciones de Postgres tengan cobertura de todas sus ramas. El resto del código se cubre según el riesgo de cada parte.

Criterios de aprobación

Un pull request se puede mergear solo si pasa lint y chequeo de tipos sin errores, todos los tests unitarios y de base de datos pasan, y los tests de flujos afectados pasan en staging. Ningún test se desactiva para hacer pasar el pipeline. Si un test está inestable, se arregla o se elimina con una justificación en el pull request.

Datos para los tests

Los tests de base de datos crean sus propios datos al inicio y los borran al final, así no dependen del orden de ejecución ni de lo que haya en staging. Los tests de flujos usan los datos del script de seed, y como ese script es idempotente, correrlo varias veces no genera duplicados.

Convenciones

Los tests se nombran describiendo el comportamiento esperado, por ejemplo “no descuenta insumos si falta alguno de la receta”, y no el nombre de la función. Cada bug que llega a staging o producción se reproduce primero con un test que falla, y después se corrige. Ese test queda en el repositorio para que el error no vuelva. El pipeline de integración y despliegue se describe en la sección 9.

9. CI/CD

El pipeline se define en `.github/workflows/` y tiene dos archivos: `ci.yml`, que corre en cada pull request, y `deploy.yml`, que corre al hacer push a `development` (staging) o a `main` (producción).

Integración (pull request)

Corre en cada pull request hacia `development` y `main` y bloquea el merge si falla.

Paso	Qué hace	Falla si
Lint y tipos	ESLint y `tsc --noEmit`	Hay errores de lint o de tipos

Tests unitarios	Vitest sobre `lib/`	Algún test falla o baja la cobertura de `lib/stock/`
Tests de base de datos	Levanta Supabase local, aplica migraciones y corre los tests de funciones y RLS	Una migración no aplica o un test de Postgres falla
Build	`next build`	El build no completa

Los pasos corren en paralelo cuando no dependen unos de otros. Las dependencias de npm se cachean para acortar los tiempos.

Despliegue a staging (merge a development)

Paso	**Entorno**	**Detalle**
1. Migraciones y deploy	Staging	Se aplican las migraciones pendientes con la CLI de Supabase y Vercel publica la versión en `gelato-track-staging.vercel.app`
2. Tests de flujos	Staging	Playwright corre los recorridos principales contra staging. Si fallan, el pipeline se detiene

Despliegue a producción (merge a main)

Paso	**Entorno**	**Detalle**
1. Aprobación	GitHub Environment `production`	El job queda pausado hasta que un revisor autorizado lo apruebe
2. Migraciones y deploy	Producción	Se aplican las migraciones pendientes en `gelato-track-prod` y Vercel publica la versión en `gelato-track.vercel.app`
3. Smoke test	Producción	Un flujo mínimo (login y consulta de stock) verifica que la versión responde bien

Si el paso 3 falla, se dispara el procedimiento de rollback de la sección 7.

Estructura del workflow

La aprobación manual no está en el YAML: se configura en el *environment* `production` de GitHub, con revisores requeridos. Como ese environment aparece en el job, GitHub pausa la ejecución hasta que se apruebe.

Cada job corre solo en su rama, con la condición `if:` sobre `github.ref`. Los secretos y variables se leen desde el environment del job.

El archivo principal de despliegue sigue esta forma general:

```yaml
name: deploy

on:

  push:

    branches: [main, development]

jobs:

  staging:

    if: github.ref == 'refs/heads/development'

    runs-on: ubuntu-latest

    environment: staging

    env:

      SUPABASE_ACCESS_TOKEN: ${{ secrets.SUPABASE_ACCESS_TOKEN }}

      SUPABASE_DB_PASSWORD: ${{ secrets.SUPABASE_DB_PASSWORD }}

    steps:

      - uses: actions/checkout@v4

      - uses: supabase/setup-cli@v1

      - run: supabase link --project-ref ${{ vars.SUPABASE_PROJECT_REF }}

      - run: supabase db push

      - uses: actions/setup-node@v4

        with:
```

```yaml
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test
        env:
          BASE_URL: ${{ vars.STAGING_URL }}
  production:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    env:
      SUPABASE_ACCESS_TOKEN: ${{ secrets.SUPABASE_ACCESS_TOKEN }}
      SUPABASE_DB_PASSWORD: ${{ secrets.SUPABASE_DB_PASSWORD }}
    steps:
      - uses: actions/checkout@v4
      - uses: supabase/setup-cli@v1
      - run: supabase link --project-ref ${{ vars.SUPABASE_PROJECT_REF }}
      - run: supabase db push --dry-run
      - run: supabase db push
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
```

```
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test tests/e2e/smoke.spec.ts
        env:
          BASE_URL: ${{ vars.PROD_URL }}
          SMOKE_USER_EMAIL: ${{ secrets.SMOKE_USER_EMAIL }}
          SMOKE_USER_PASSWORD: ${{ secrets.SMOKE_USER_PASSWORD }}
```

Protección de ramas y secretos

- `development` y `main` tienen protección: no admiten push directo, requieren pull request y deben pasar los checks de integración (lint, typecheck, test y build). Ninguna de las dos admite force push ni borrado.
- `development` requiere 0 aprobaciones humanas, como se define en ADR-011. `main` tiene la aprobación manual del environment `production` (ADR-009).
- Los secretos de cada entorno están asociados al *environment* correspondiente. Los de producción solo pueden usarse desde el job de producción, así que un job de staging no puede leerlos.
- Dependabot abre pull requests semanales para actualizar dependencias. Esos PR apuntan a `development` y pasan por el mismo pipeline.

Métricas que se revisan

Se mira el tiempo del pipeline de integración (objetivo: menos de 10 minutos), la tasa de fallos de Playwright en staging y la cantidad de rollbacks. Un test que falla de forma intermitente se considera un bug del pipeline y se corrige antes de sumar funcionalidades nuevas.

10. Decisiones de arquitectura (ADR)

ADR-001: Next.js con TypeScript para la aplicación

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: se necesita una aplicación web con formularios, páginas de reportes y lógica de servidor, que una sola persona pueda mantener.
- Decisión: Next.js con App Router y TypeScript en un único repositorio.
- Consecuencias: el frontend y las acciones del servidor comparten tipos. Se depende del ecosistema de Next.js y de sus cambios de versión.

ADR-002: Supabase como plataforma de datos, autenticación y autorización

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el proyecto necesita Postgres, login y control de acceso por rol, sin tiempo para construir infraestructura propia.
- Decisión: Supabase con Postgres, Supabase Auth y Row Level Security.
- Consecuencias: las reglas de acceso viven en la base y se pueden testear. Hay dependencia de un proveedor, y las funciones avanzadas de Postgres quedan atadas a su versión.

ADR-003: Operaciones críticas como funciones de Postgres

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: registrar una tanda, una venta o un cierre de caja modifica varias tablas. Hacerlo en varias llamadas desde la aplicación puede dejar datos a medio guardar.
- Decisión: `registrar_tanda`, `registrar_venta` y `cerrar_turno` son funciones de Postgres que se ejecutan en una transacción.
- Consecuencias: la lógica de negocio más sensible queda en SQL, y hay que testearla con Vitest contra Supabase local. A cambio, no hay estados intermedios inconsistentes.

ADR-004: Sin backend separado en la versión 1

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: no hay clientes externos que consuman una API, y mantener un servicio aparte duplicaría el trabajo.
- Decisión: la lógica de aplicación vive en acciones del servidor de Next.js y en las funciones de Postgres.
- Consecuencias: menos superficie de ataque y menos mantenimiento. Si más adelante aparece una app móvil o integraciones externas, habrá que exponer una API, y eso se decidirá en su propio ADR.

ADR-005: Anulaciones sin borrado físico

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el stock y la caja necesitan trazabilidad para auditar diferencias.
- Decisión: ningún registro de tandas, ventas o movimientos se borra. Una anulación marca el estado, guarda quién y cuándo, y crea movimientos de reversa.
- Consecuencias: las tablas crecen más, pero el historial es completo. Los reportes deben filtrar operaciones anuladas explícitamente.

ADR-006: Recetas versionadas

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: cambiar una receta no debe alterar el costo ni el consumo de tandas ya producidas.
- Decisión: cada cambio de receta crea una versión nueva con fecha de vigencia. Cada tanda referencia la versión con la que se produjo.
- Consecuencias: la tabla `recetas` tiene más filas, y la consulta de la receta vigente debe considerar la fecha.

ADR-007: Selección de lotes en orden FEFO

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el helado tiene vencimiento, y vender lotes nuevos primero genera mermas.
- Decisión: al vender, se descuenta primero de los lotes que vencen antes. No se vende de lotes vencidos.
- Consecuencias: la lógica de selección debe estar en `lib/stock/` con tests exhaustivos, porque es donde más se puede equivocar.

ADR-008: Dos proyectos de Supabase, staging y producción

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: hay que probar migraciones y despliegues sin tocar datos reales.
- Decisión: dos proyectos independientes. Las migraciones se aplican primero en staging. Local usa Supabase con Docker.
- Consecuencias: hay que mantener sincronizadas las migraciones entre entornos, y el pipeline se encarga de eso.

ADR-009: Aprobación manual para desplegar a producción

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: un error en producción afecta el stock real de la heladería.
- Decisión: el job de producción está detrás de un *environment* de GitHub con revisores requeridos.
- Consecuencias: cada despliegue a producción es una decisión consciente, a costa de un paso manual.

ADR-010: Presentaciones y precios

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el modelo no registraba lo que se vende (vasito, cucurucho, cuarto, kilo) ni sus precios.
- Decisión: tabla `presentaciones` con sus litros equivalentes, tabla `precios` por sabor y presentación con historial por `vigente_desde`, y `venta_items` que guarda el precio del momento de la venta.
- Consecuencias: el descuento FEFO sale de `cantidad × litros_equivalentes`. La equivalencia del kilo (1 kg = 1 l) es provisional y se revisa con un factor de conversión si hace falta.

ADR-011: Estrategia de ramas y pull requests

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el proyecto lo desarrolla una sola persona, pero el flujo tiene que ser el de un equipo real.
- Decisión: ramas cortas por tarea (`feat/`, `fix/`, `chore/`, `ci/`, `test/`, `docs/`) que salen de `development` y vuelven a `development` por pull request. Todo cambio entra por pull request con CI obligatorio y *Squash and merge*. La aprobación de revisor está en 0 porque GitHub no permite aprobar el propio PR. El modelo completo de ramas está en ADR-015.
- Consecuencias: el historial de `development` tiene un commit por tarea, y `main` recibe un commit por cada promoción. La revisión humana queda reemplazada por el CI y por la aprobación manual de producción (ADR-009).

ADR-012: Convención de commits

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el historial tiene que ser legible y el título de la PR se convierte en el mensaje del commit al hacer *Squash and merge*.
- Decisión: Conventional Commits, con tipos como `chore`, `ci`, `test`, `fix` y `feat`, seguidos del número de PR entre paréntesis.
- Consecuencias: el título de la PR se trata como parte del historial y se revisa antes de mergear.

ADR-013: Funciones SECURITY DEFINER para operaciones de negocio

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: el linter de Supabase reporta como advertencia las funciones SECURITY DEFINER ejecutables por usuarios autenticados.
- Decisión: las operaciones de negocio y la consulta de stock por sabor son SECURITY DEFINER, porque RLS no permite escritura directa y el vendedor no puede leer lotes. Cada función valida el rol o la propiedad del registro antes de operar y tiene `search_path` fijo. Las funciones de trigger no tienen permiso de EXECUTE. `rol_actual()` se mantiene SECURITY DEFINER para evitar recursión con la política de `usuarios`.
- Consecuencias: el linter mantiene advertencias intencionales. Toda función nueva SECURITY DEFINER tiene que validar permisos dentro de su cuerpo.

ADR-014: Migraciones inmutables una vez aplicadas en producción

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: editar una migración ya aplicada hace que el repo y la base real no coincidan.
- Decisión: hasta que una migración llegue a producción puede editarse, con reset del entorno de prueba. A partir de ese punto, todo cambio es una migración nueva. La verificación se hace con `supabase migration list` y la consulta a `supabase_migrations.schema_migrations`.
- Consecuencias: el historial de producción siempre coincide con el repo, a costa de migraciones correctivas cuando hay errores.

ADR-015: Modelo de ramas con integración separada de producción

- Estado: aceptada. Fecha: 2026-10-27.
- Contexto: un push a main desplegaba a staging y producción sin separar la integración de lo que está en producción. Además, el deploy automático de Vercel desde main saltaba la aprobación manual del environment production.
- Decisión: development es la rama de integración y su deploy va a staging. main es producción y su deploy requiere aprobación. Las ramas de tarea (feat/, fix/, chore/, ci/, test/, docs/) salen de development y vuelven a development por PR con Squash and merge. La promoción a producción es un PR de development a main. Los hotfixes salen de main, se mergean a main y luego main se trae a development.
- Consecuencias: hay un paso más antes de producción, a cambio de que staging siempre refleje lo integrado y de que producción nunca reciba código sin pasar por staging. Ambas ramas tienen protección con checks de CI.

11. Registro de cambios

Versión del producto	Fecha	Cambios
0.1.0	27-10-2026	Borrador inicial: visión, alcance, roles, modelo de datos (tablas, relaciones y tipos), diagrama UML, flujos principales, arquitectura, entornos, calidad, CI/CD y ADR.
0.2.0	27-10-2026	Modelo de ramas `development` y `main` (ADR-015). Entornos con dominios definitivos. Secretos organizados por alcance. Modelo de datos actualizado (presentaciones, precios, anulaciones, movimientos). Pipeline separado en su propia sección. ADR-010 a ADR-015 incorporados. Nombres de proyectos definitivos.

Pendientes para las siguientes versiones: definir el diseño de pantallas, cargar el script de datos de prueba y completar las validaciones de cada flujo cuando empiece la implementación.

12. Glosario

Término	Definición
Tanda	Producción de un sabor en una fecha determinada. Descuenta insumos según la receta vigente y genera uno o más lotes.
Lote	Porción de una tanda con su fecha de vencimiento y los litros que quedan disponibles.
Receta vigente	Versión de la receta de un sabor que se usa para producir hoy. Las versiones anteriores se conservan para el historial.
Insumo	Materia prima que consume una receta, como leche, azúcar o pulpa de fruta.
Stock mínimo	Cantidad por debajo de la cual se genera una alerta para reponer un insumo o un sabor.
Stock disponible	Litros de un sabor que suman todos sus lotes no vencidos.

FEFO	*First Expired, First Out*: se vende primero el lote que vence antes.
Merma	Producto que se pierde por vencimiento, rotura o descarte.
Turno de caja	Período en que un vendedor tiene la caja abierta, desde la apertura hasta el cierre con monto contado.
Diferencia de caja	Resta entre el monto contado al cerrar el turno y el monto esperado según las ventas.
Movimiento	Registro de un cambio de stock, ya sea consumo, venta, ajuste o reversa. Nunca se borra.
Anulación	Marca una operación como anulada y genera movimientos de reversa. No borra el registro original.
Rol	Conjunto de permisos asignado a un usuario: admin, encargado o vendedor.
RLS	*Row Level Security*: políticas de Postgres que restringen qué filas puede leer o modificar cada usuario.
Preview	Entorno temporal que Vercel genera para cada pull request.
Staging	Entorno de pruebas previo a producción, con su propia base de datos y datos de prueba.
Pull request	Propuesta de cambio en el repositorio que se revisa y se prueba antes de integrarse a `main`.
Migración	Archivo versionado que modifica el esquema de la base de datos.
ADR	*Architecture Decision Record*: documento corto que registra una decisión técnica, su contexto y sus consecuencias.
Smoke test	Prueba mínima que verifica que una versión recién desplegada responde y funciona en sus caminos principales.

Development	Rama de integración del repositorio. Recibe las tareas por pull request y despliega a staging.
Environment (GitHub)	Entorno de GitHub con sus propios secrets, variables y reglas de aprobación. Los usamos `staging` y `production`.
Main	Rama de producción. Solo recibe cambios desde `development` mediante pull request de promoción.
Rama de tarea	Rama corta (`feat/`, `fix/`, `chore/`, etc.) que sale de `development` para una tarea y vuelve a `development` por pull request.
Squash and merge	Forma de mergear un pull request que lo convierte en un solo commit en la rama destino.